SENTINEL Lab 1 Primer: `make`, tests and `/proc`

ECE 476 Engineering System Software • Fall 2026 • University of Victoria

This primer gives the basic knowledge for four parts of Lab 1: your `Makefile`, your tests, the wrapper functions, and `proc.c`. It has no marks, and it adds no rule. The Lab 1 Brief[1] and `syscall.h`[2] state the contract. Run each command of this primer in the top directory of your repository.

1 `make` and your Makefile

`make` builds files from rules. It reads the file `Makefile` in the current directory. `make <target>` runs the rule for that target, and `make` alone runs the first rule of the file. The examples below come from two files that you have: `Makefile` (the top-level Makefile) and `common/Makefile`.

1.1 A rule

```
$(BUILD):
        mkdir -p $(BUILD)
```

A rule has a **target**, a colon, the **prerequisites**, and on the next lines the **recipe**. The target is the file that the rule makes. The prerequisites are the files that the target needs; this rule has none. The recipe is the shell commands. **Each recipe line starts with a TAB character.** Spaces give an error that starts with `Makefile:2: *** missing separator`. `make` runs a recipe only when the target does not exist, or when a prerequisite is newer than the target. So a second `make` compiles only the files that you changed.

1.2 Variables

```
CC      := gcc
PROFILE ?= world
export CC CFLAGS LDFLAGS BUILD
```

- `NAME := value` sets a variable. `$(NAME)` gives its value.
- `NAME ?= value` sets it only when it has no value yet.
- A value on the command line replaces the value in the file: `make lab1 PROFILE=dirty`.[3]
- `export` gives the variables to each `make` that this one starts, such as the `make` of your lab: `$(MAKE) -C src/libsyscall` runs `make` in that directory. In your Makefile, `CC := gcc` replaces that value, and `CC ?= gcc` keeps it. So the Brief, Section 4.1, asks for `?=`.

1.3 Patterns and automatic variables

```
$(LIB): $(OBJS)
        ar rcs $@ $^

$(BUILD)/common/%.o: %.c $(HDRS) | $(BUILD)/common
        $(CC) $(CFLAGS) -c -o $@ $<
```

Item	What it means
`$@`, `$<`, `$^`	the target; the first prerequisite; all the prerequisites
`%`	a pattern: the same text in the target and in the prerequisite. One rule makes `contact.o` from `contact.c`, and each other `.o` from its `.c`
`| $(BUILD)/common`	this prerequisite must exist before the recipe runs, but its time does not cause a new build. Use it for a directory
`gcc -c -o x.o x.c`	compile one source file to one object file, with no link
`ar rcs lib.a a.o b.o`	put the object files into a static library

Your library needs the same two rules, with `src/%.c` as the prerequisite, `include/sentinel/syscall.h` as the header, and `-Iinclude -I../../common` in the compile command. `-I` adds a directory in which the compiler finds the files of `#include`. Put the object files in a directory of their own, for example `ODIR := $(BUILD)/libsyscall`.

A program has no extension. One pattern rule, `$(BUILD)/%: tests/%.c $(LIB)`, makes `$(BUILD)/demo_lab1` and each `$(BUILD)/test_*`. Its recipe is `$(CC) $(CFLAGS) -Iinclude -I../../common -o $@ $<`, then the three li-

[1] The Lab 1 Brief, `docs/lab1/lab1_brief.pdf` in your repository. It states the contract of Lab 1.

[2] `src/libsyscall/include/sentinel/syscall.h`, in your repository. It states what each function of Lab 1 does, and each of its errors.

[3] `make lab1` writes a new feed, `build/live/feed.txt`, from the four sensors, and it runs your demo on it. The SENTINEL Lab Manual, Section 4, `docs/manual/sentinel_manual.pdf`, lists each make target.

braries in the order that the Brief[4] gives in Section 4.1. The linker reads from left to right, so a library comes after the file that uses it. `-lm` is the mathematics library.

Three functions give lists of files:

- `$(wildcard src/*.c)` gives each file on the disk that matches. The top-level Makefile uses `$(wildcard $(d)/Makefile)`. `*` is a shell pattern: any text.
- `$(patsubst src/%.c,$(ODIR)/%.o,$(SRCS))` changes each name in a list: `src/io.c` becomes `$(ODIR)/io.o`. Here `%` is the pattern of `make`. Use `*` to find files on the disk, and `%` to change a name or to write a pattern rule.
- `$(abspath ../../build)` gives the absolute path of a directory.

1.4 Recipes are shell commands

```
test: all
	@for d in $(READY); do $(MAKE) -C $$d test || exit 1; done
```

- `.PHONY: all test ...` names the targets that are not files. Without it, if a file `test` exists, `make` prints `make: 'test' is up to date.` and does not run the rule.
- `@` at the start of a line: `make` does not print the command.
- `$$d`: `make` changes `$$` to `$`, so the shell sees `$d`. One `$` is for `make`.
- `|| exit 1`: when one command fails, the loop stops, and `make` stops. Your `test` target uses this to stop at the first failure.
- Each recipe line runs in a new shell. Put a loop on one line, or end each line with `\`.

1.5 What your Makefile needs

Brief, Section 4.1	What you use
1. `$(BUILD)/libsyscall.a`	`$(wildcard src/*.c)`, `patsubst` to the names in `$(ODIR)`, the object rule, `ar rcs`, and a rule with `mkdir -p $(ODIR)`
2. `$(BUILD)/demo_lab1`	the program rule, and a first rule `all: $(LIB) $(BUILD)/demo_lab1`: the top level runs your `make` with no target
3. the `test` target	`TESTS`, the names `$(BUILD)/test_*` from `$(wildcard tests/test_*.c)` and `patsubst`; then `test: $(TESTS)`, and a recipe `for t in $(TESTS); do $$t || exit 1; done`
4. the `clean` target	`rm -rf` of each file and directory that your rules made. Run it with `make -C src/libsyscall clean`: the top-level `make clean` removes all of `build/`

The top-level `make` also builds the given code of the later labs, and prints a line for each lab that is not complete. That is normal. `make -n` prints the commands and runs none. To read more, use the GNU make manual, https://www.gnu.org/software/make/manual/make.html, first "An Introduction to Makefiles".

2 Tests and `assert`

A test is a C program. It calls your functions and checks each result with `assert`. When each check is true, it returns 0. `make test`[5] runs it.

`assert(expression)` comes from `<assert.h>`. When the expression is false, it prints the file, the line, the function and the expression, and it stops the program. The shell prints `Aborted`, and the exit status is 134. So `make` sees a failure. For example, with `ENOENT` changed to `EACCES` in the test below:

```
test_device: /tmp/test_device.c:25: missing_then_file: Assertion `errno == EACCES' failed.
Aborted (core dumped)
```

Do not put a call that does work inside `assert`. With `-DNDEBUG`, the compiler removes each `assert` and the call in it.

The example below tests the promises of `common/device.h`, the smallest given file. It is an example only: the Brief, Section 4.2, tells you not to test `common/`. Your four tests have the same form, and test the promises and errors of `syscall.h`.[6]

[4] The Lab 1 Brief, `docs/lab1/lab1_brief.pdf` in your repository. It states the contract of Lab 1.

[5] `make test` runs the tests of each lab that has a `Makefile`. The SENTINEL Lab Manual, Section 4, `docs/manual/sentinel_manual.pdf`, lists each make target.

[6] `src/libsyscall/include/sentinel/syscall.h`, in your repository. It states what each function of Lab 1 does, and each of its errors.

```c
/* Example: the promises of common/device.h. */
#include "device.h"

#include <assert.h>
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>

/* A NULL path gives -1 and EINVAL. */
static void null_path(void)
{
    errno = 0;
    int fd = device_open(NULL, 0);
    assert(fd == -1);
    assert(errno == EINVAL);
}
```

To try it, save it as `/tmp/test_device.c`, run `make`, then:

```
$ gcc -std=c11 -D_GNU_SOURCE -g \
    -Wall -Wextra -Werror -Icommon \
    -o /tmp/test_device \
    /tmp/test_device.c build/libcommon.a
$ /tmp/test_device
test_device: OK
```

```c
/* A failed open gives -1 and its errno.
   After the error, a file still opens. */
static void missing_then_file(void)
{
    int fd = device_open("/tmp/no_such_device", 0);
    assert(fd == -1);
    assert(errno == ENOENT);

    const char *path = "/tmp/test_device.txt";
    int w = open(path, O_WRONLY | O_CREAT, 0600);
    assert(w >= 0);
    close(w);
    fd = device_open(path, 0);
    assert(fd >= 0);
    close(fd);
    unlink(path);
}

int main(void)
{
    null_path();
    missing_then_file();
    printf("test_device: OK\n");
    return 0;
}
```

The demo and the tests link with the whole library. Until each of the 15 functions exists, the link fails with an `undefined reference` error that names a missing function. So first write a stub of each function: it sets `errno` to `ENOSYS` and returns -1 (or `NULL`). Write each test before its code (test-driven development). Take one promise or one error of `syscall.h`,[7] write a test function for it, and see `make test`[8] fail. Then write the code until the test passes, and go to the next promise. To read more, run `man 3 assert` and `man 3 errno`.

3 Wrapper functions

A wrapper function calls another function and adds work around it. In Lab 1, each `sc_` function of `io.c` wraps one system call. `syscall.h` states what each wrapper adds.

Wrapper	It calls	It adds
`sc_open_read`	`open()`	a new call after `EINTR`
`sc_read_full`	`read()`	a loop until `n` bytes arrive or the input ends
`sc_write_all`	`write()`	a loop until each byte is written
`sc_close`	`close()`	`EINTR` is not a failure

For example, `read(fd, buf, 4096)` can return 1000 bytes when 4096 are in the file, but `sc_read_full` gives fewer only at the end of the input. Call the wrappers in `log.c` and `proc.c`, in place of the system calls.

4 /proc and proc.c

Linux shows each process that runs as a directory, `/proc/<pid>`. The kernel makes its files at the time that you read them, and they are not on a disk. `/proc/self` is the directory of the process that reads it.

4.1 The file status

```
$ grep -E 'Name|PPid|Vm(Size|RSS)|Threads|ctxt' /proc/self/status
Name:        grep
PPid:        99291
VmSize:            3468 kB
VmRSS:             1732 kB
Threads:        1
voluntary_ctxt_switches:        0
nonvoluntary_ctxt_switches:        0
```

Each line is a key, a colon, spaces or tabs, and the value. The comment of `proc_info_t` in `syscall.h` gives the key of each field. The keys are the same on aarch64 and on x86_64.

[7] `src/libsyscall/include/sentinel/syscall.h`, in your repository. It states what each function of Lab 1 does, and each of its errors.

[8] `make test` runs the tests of each lab that has a `Makefile`. The SENTINEL Lab Manual, Section 4, `docs/manual/sentinel_manual.pdf`, lists each make target.

4.2 Where open_fds comes from

/proc/<pid>/fd is a directory. It has one entry for each open descriptor of the process. The name of an entry is the descriptor number, and the entry points to the open file.

```
$ ls -la /proc/self/fd
total 0
dr-x------ 2 linux linux  0 Oct 22 05:24 .
dr-xr-xr-x 9 linux linux  0 Oct 22 05:24 ..
lrwx------ 1 linux linux 64 Oct 22 05:24 0 -> /dev/pts/1
lrwx------ 1 linux linux 64 Oct 22 05:24 1 -> /dev/pts/1
lrwx------ 1 linux linux 64 Oct 22 05:24 2 -> /dev/pts/1
lr-x------ 1 linux linux 64 Oct 22 05:24 3 -> /proc/98364/fd
```

- 0, 1 and 2 are the standard input, output and error of ls. Here they point to the terminal.
- . and .. are in each directory. ls -a shows them, and readdir also gives them. They are not descriptors, so do not count them.
- 3 points to this directory: 98364 is the pid of ls, the process that read /proc/self. ls opened it to read the directory. opendir in your proc_report also opens one. So when the process is the caller, do not count it.

So here open_fds is 3. To check it exactly, your test counts /proc/self/fd itself with opendir, and compares. It compares name with /proc/self/comm. A test causes each error in this way:

Error	A call that causes it
ENOENT	proc_report(INT_MAX, &info): no pid is that large
EACCES	proc_report(1, &info): root owns pid 1. Skip it when getuid() is 0
a failed write	proc_report_write(-1, &info) gives EBADF; a descriptor of /dev/full gives ENOSPC

4.3 The steps

Use your library where it has a function, and the C library only where it has none. To read about a function of the C library, use man 3, for example man 3 opendir. To check your work, run make test,[9] then the acceptance test[10] (make accept1), which has 6 checks of proc.c.

proc_report(pid, out):

1. If out is NULL, set errno to EINVAL and return -1. Then set each field of out to 0 (memset).
2. If pid is 0, the directory is /proc/self, and out->pid is getpid(). If not, the directory is /proc/<pid>, and out->pid is pid.
3. Open <directory>/status with sc_open_read. If it fails, return -1: errno is already ENOENT when no process has that pid.
4. Read the file into a buffer of 4096 bytes with sc_read_full. Ask for one byte less than the buffer. If it fails, save errno, close the file, put errno back, and return -1.
5. Close the file now: if it is still open at step 8, the count includes it. Put '\0' after the last byte read.
6. Divide the buffer into lines at each '\n' (strchr). A line holds a key when strncmp finds the key at its start, and the next character is ':'. Skip the spaces and tabs after it. strtol reads a number and stops at " kB". Copy the name with snprintf into name: it stops at 63 characters and adds the '\0'.
7. Open <directory>/fd with opendir. If it fails, return -1: errno is EACCES for a process of another user.
8. Count each entry that readdir gives, except . and ... Set errno to 0 before each readdir: it gives NULL at the end and on an error, and only errno shows which. Then call closedir.
9. Put the count in out->open_fds, minus 1 when the process is the caller (pid is 0 or getpid()). Return 0.

proc_report_write(fd, info): if info is NULL, set errno to EINVAL and return -1. Then write the whole report into one buffer of 512 bytes with one snprintf, in the layout of syscall.h.[11] A label left-aligned in 10 characters is %-10s. Write the buffer with sc_write_all, and return its result.

[9] make test runs the tests of each lab that has a Makefile. The SENTINEL Lab Manual, Section 4, docs/manual/sentinel_manual.pdf, lists each make target.

[10] The acceptance test of Lab 1, make accept1, runs 34 checks of your library and your demo. The Lab 1 Brief, Section 4.3, docs/lab1/lab1_brief.pdf, describes it.

[11] src/libsyscall/include/sentinel/syscall.h, in your repository. It states what each function of Lab 1 does, and each of its errors.